

Introducción a la programación distribuida

con MPICH

Navajas Román, Ángel - i02naroa
■ Sanchez Gutierrez, Daniel - 12sagud

ÍNDICE DE CONTENIDOS

01 INTRODUCCIÓN

02 PLANTEAMIENTO DEL PROBLEMA

03 METODOLOGÍA

04 RESULTADOS

05 DISCUSIÓN

06 CONCLUSIÓN

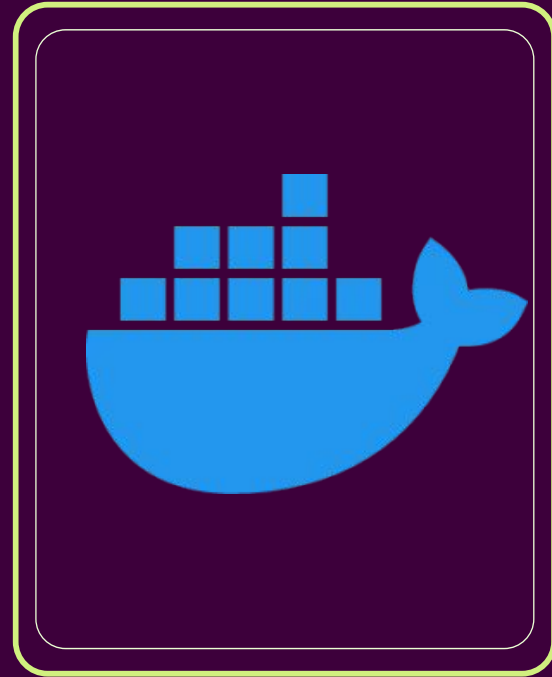
INTRODUCCIÓN

Docker, puesta en marcha
del cluster y Git

01

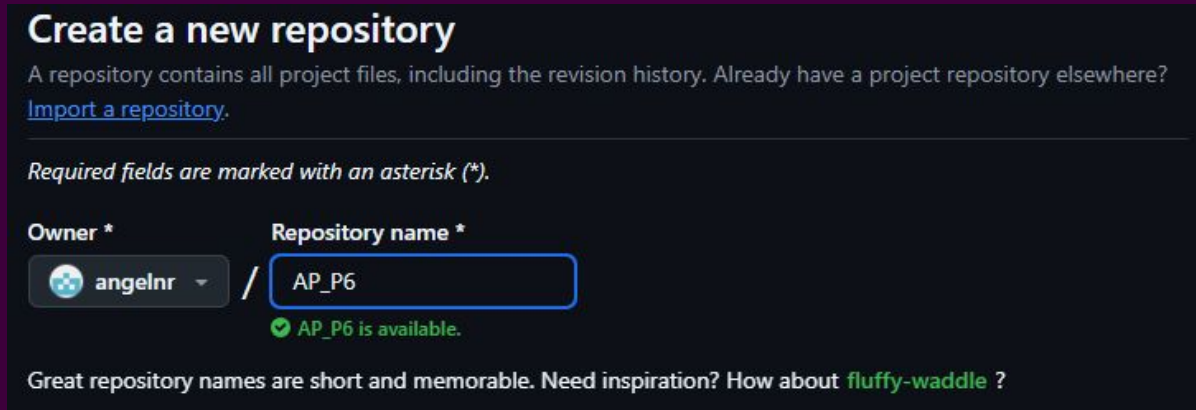
Docker

- 1) `sudo apt install docker.io docker`
`docker-buildx docker-compose-v2.`
- 2) `sudo apt install git`
- 3) `git clone -b new`
`git@github.com:gentooza/alpine-mpich.git`
- 4) `docker build --build-arg`
`MPICH_VERSION="4.0.3" -t`
`nlknnguyen/alpine-mpich base/`
- 5) `docker build -t`
`nlknnguyen/alpine-mpich:onbuild onbuild/.`



Git y GitHub. Configuración

- Clonamos el repositorio de la práctica
 - `git clone -b new`
<https://github.com/gentooza/alpine-mpich.git>
- Creamos un repositorio en github
- Añadimos el repositorio remoto
 - `git remote add origin_angel`
`https://github.com/angelnr/AP_P6_MPICH.git`



Create a new repository

A repository contains all project files, including the revision history. Already have a project repository elsewhere? [Import a repository.](#)

Required fields are marked with an asterisk ().*

Owner * **Repository name ***

 **angelnr** /

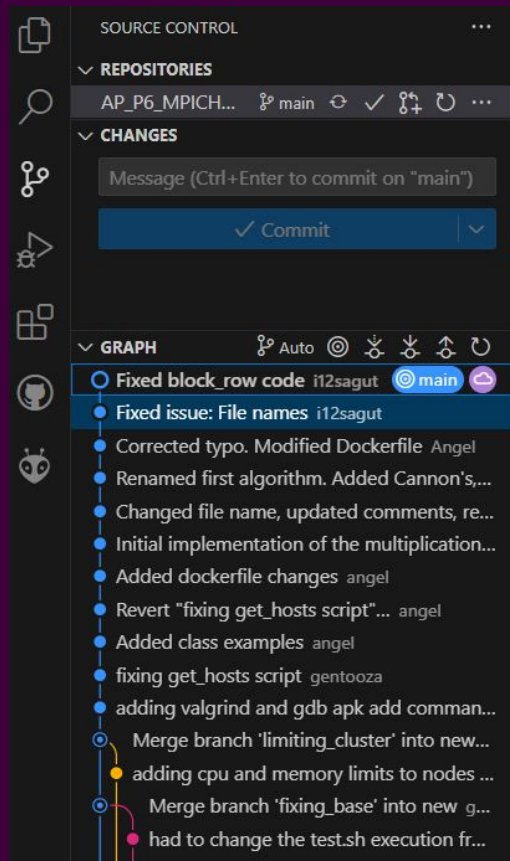
✓ AP_P6 is available.

Great repository names are short and memorable. Need inspiration? How about [fluffy-waddle](#) ?

Git y GitHub. Configuración

- Eliminamos el anterior repositorio remoto y cambiamos el predeterminado (Opcional)
 - `git remote remove origin`
 - `git remote rename origin_angel origin`
- Realizamos cambios a nuestro código
- Subimos los cambios desde local al repositorio con: `git push -u origin-tu-cuenta main`.
 - `git add .`
 - `git push -u origin-tu-cuenta main`

Git y GitHub. Configuración



Equipo

Especificaciones técnicas:

- **Intel(R) Core(TM) i7-9750H CPU @ 2.60GH.**
- Tamaño de caché de nivel 1
 - 6 x 32 KB caché de instrucciones
 - 6 x 32 KB caché de datos.
- Tamaño de caché de nivel 2
 - 6 x 256 KB
- Tamaño de caché de nivel 3
 - 12MB



PLANTEAMIENTO DEL PROBLEMA

02

Planteamiento del problema

Propuesta:

- Diseño e implementación de un algoritmo distribuido utilizando la biblioteca MPICH para paralelizar matrices $N \times N$ (siendo N múltiplo de 8).

Planteamiento:

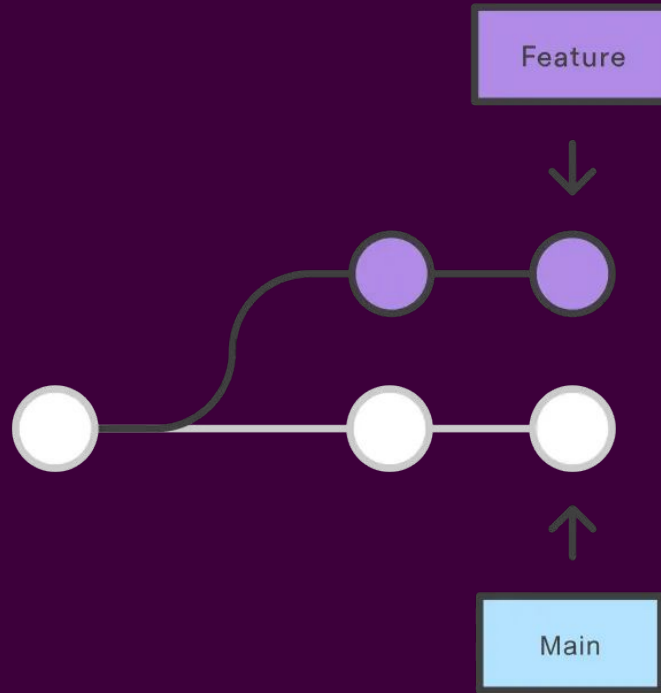
- Implementar una solución eficiente en C
- Comparar distintos algoritmos.
- Evaluar el rendimiento mediante métricas de tiempo y speed-up.
- Discutir limitaciones y mejoras potenciales.

METODOLOGÍA

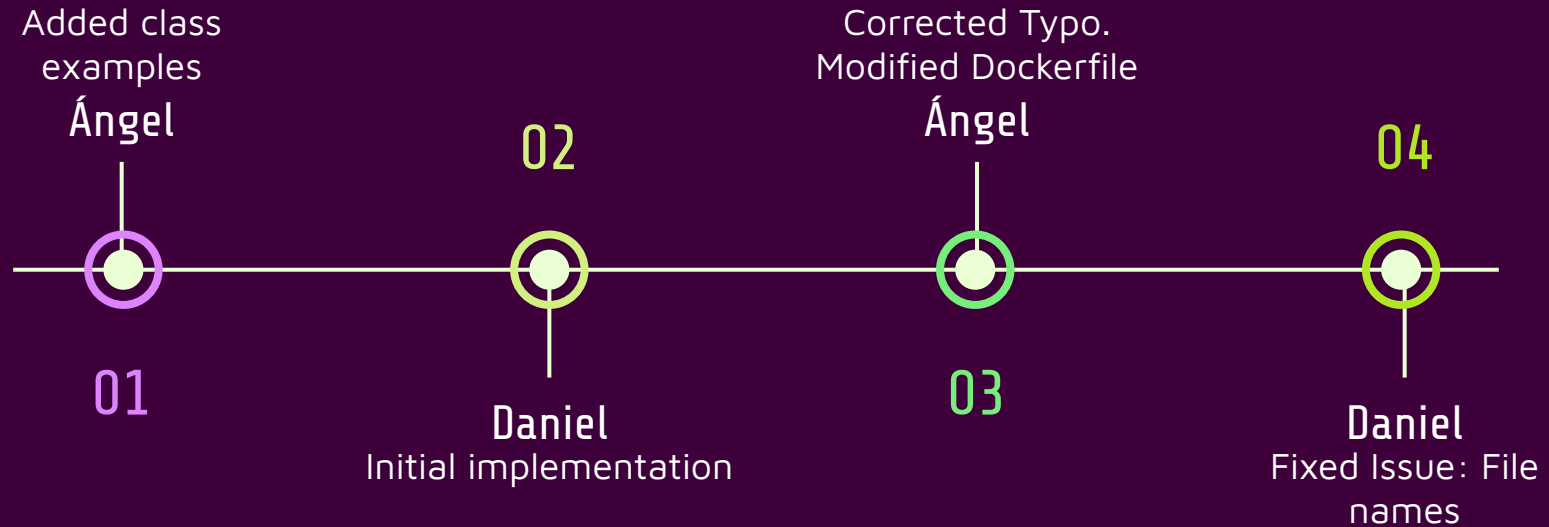
Flujo de trabajo con Git,
explicación de algoritmos,
temporización

03

Flujo de trabajo con Git



Flujo de trabajo con Git



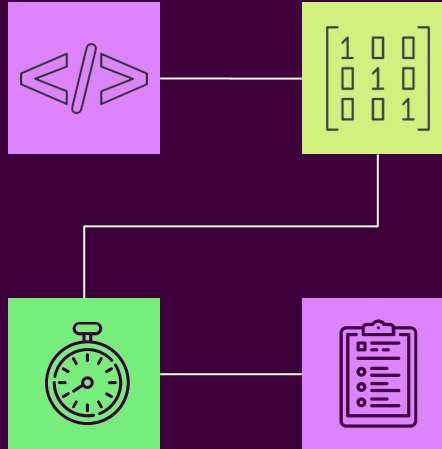
Algoritmos

Block Row

Divide por filas,
distribuye el trabajo

Algoritmo de Fox

Difusión inteligente de
bloques diagonales.



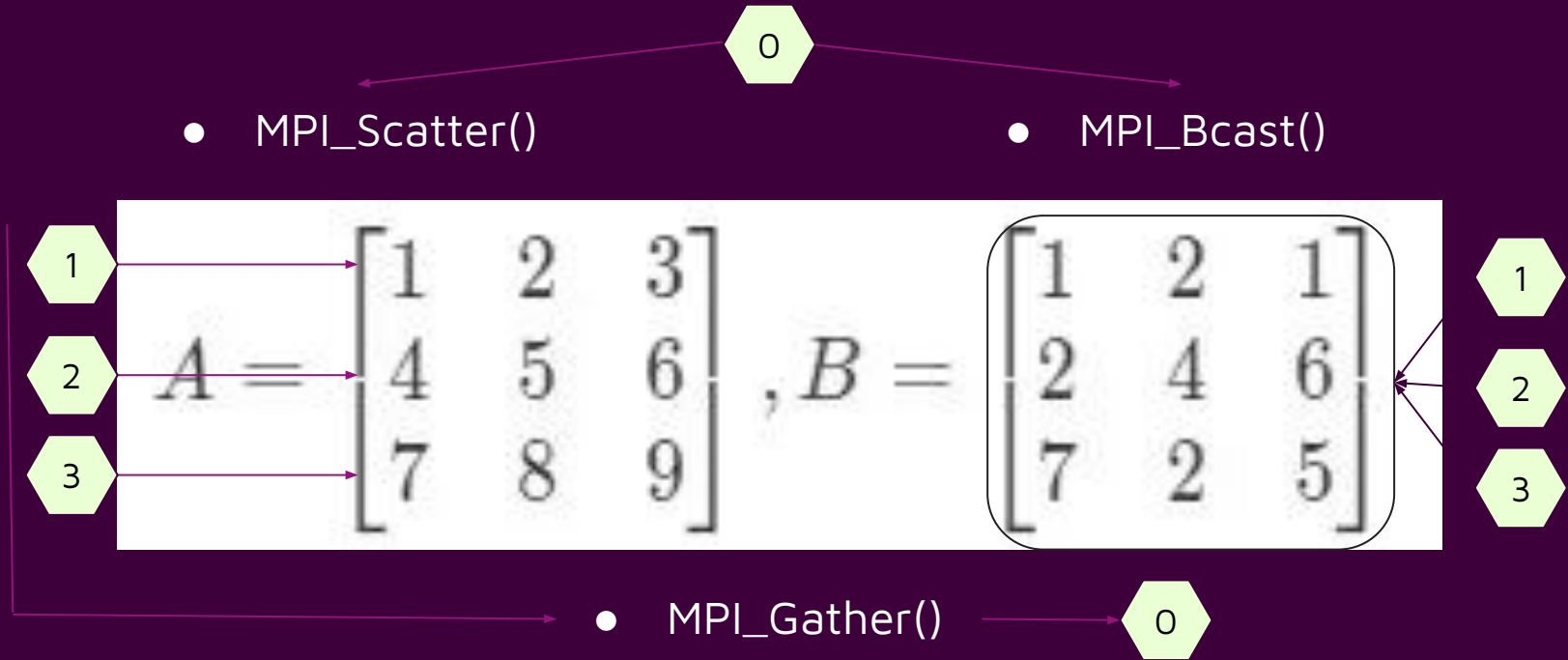
Algoritmo de Cannon

Rotación de bloques
para minimizar
comunicación

Algoritmo de Strassen

Ahorra operaciones de
multiplicación

Block Row. Ejemplo



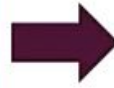
Algoritmo de Cannon. Ejemplo

MATRIZ A

4	3	6	3
7	7	4	9
8	1	2	5
3	5	8	5

MATRIZ B

4	2	4	8
1	3	5	7
2	4	8	6
5	7	3	8



MATRIZ A

A_{00}	A_{01}	A_{02}	A_{03}
A_{10}	A_{11}	A_{12}	A_{13}
A_{20}	A_{21}	A_{22}	A_{23}
A_{30}	A_{31}	A_{32}	A_{33}

MATRIZ B

B_{00}	B_{01}	B_{02}	B_{03}
B_{10}	B_{11}	B_{12}	B_{13}
B_{20}	B_{21}	B_{22}	B_{23}
B_{30}	B_{31}	B_{32}	B_{33}

MATRIZ RESULTADO

C1	C2	C3	C4
C5	C6	C7	C8
C9	C10	C11	C12
C13	C14	C15	C16

Algoritmo de Cannon. Ejemplo

MATRIZ A

A_{00}	A_{01}	A_{02}	A_{03}
A_{10}	A_{11}	A_{12}	A_{13}
A_{20}	A_{21}	A_{22}	A_{23}
A_{30}	A_{31}	A_{32}	A_{33}

← 1 izquierda

← 2 izquierda

← 3 izquierda

A_{00}	A_{01}	A_{02}	A_{03}
A_{11}	A_{12}	A_{13}	A_{10}
A_{22}	A_{23}	A_{20}	A_{21}
A_{33}	A_{30}	A_{31}	A_{32}

Algoritmo de Cannon. Ejemplo

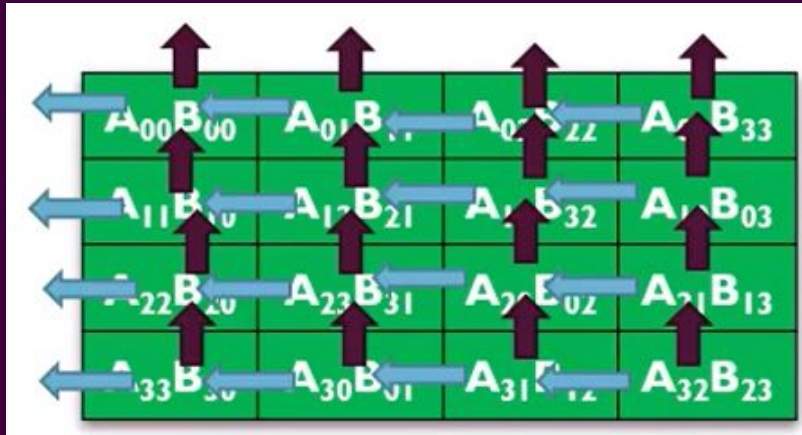
MATRIZ B			
B_{00}	B_{01}	B_{02}	B_{03}
B_{10}	B_{11}	B_{12}	B_{13}
B_{20}	B_{21}	B_{22}	B_{23}
B_{30}	B_{31}	B_{32}	B_{33}
	↑ 1 arriba	↑ 2 arriba	↑ 3 arriba

B_{00}	B_{11}	B_{22}	B_{33}
B_{10}	B_{21}	B_{32}	B_{03}
B_{20}	B_{31}	B_{02}	B_{13}
B_{30}	B_{01}	B_{12}	B_{23}

Algoritmo de Cannon. Ejemplo

				MATRIZ RESULTADO			
C1	C2	C3	C4	$A_{00}B_{00}$	$A_{01}B_{11}$	$A_{02}B_{22}$	$A_{03}B_{33}$
C5	C6	C7	C8	$A_{11}B_{10}$	$A_{12}B_{21}$	$A_{13}B_{32}$	$A_{10}B_{03}$
C9	C10	C11	C12	$A_{22}B_{20}$	$A_{23}B_{31}$	$A_{20}B_{02}$	$A_{21}B_{13}$
C13	C14	C15	C16	$A_{33}B_{30}$	$A_{30}B_{01}$	$A_{31}B_{12}$	$A_{32}B_{23}$

Algoritmo de Cannon. Ejemplo



MATRIZ RESULTADO			
4*4	3*3	6*8	3*8
3*1	6*4	3*3	4*8
6*2	3*7	4*4	3*7
3*5	4*2	3*5	6*6
7*1	4*4	9*3	7*8
4*2	9*7	7*4	7*7
9*5	7*2	7*5	4*6
7*4	7*3	4*8	9*8
2*2	5*7	8*4	1*7
5*5	8*2	1*5	2*6
8*4	1*3	2*8	5*8
1*1	2*4	5*3	8*8
5*5	3*2	5*5	8*6
3*4	5*3	8*8	5*8
5*1	8*4	5*3	3*8
8*2	5*7	3*4	5*7

Algoritmo de Fox. Ejemplo

P_{00}			P_{01}			P_{02}		
A_{00}	$A_{00} \times B_{00}$		A_{01}	$A_{00} \times B_{01}$		A_{02}	$A_{00} \times B_{02}$	
P_{10}			P_{11}			P_{12}		
A_{10}	$A_{11} \times B_{10}$		A_{11}	$A_{11} \times B_{11}$		A_{12}	$A_{11} \times B_{12}$	
P_{20}			P_{21}			P_{22}		
A_{20}	$A_{22} \times B_{20}$		A_{21}	$A_{22} \times B_{21}$		A_{22}	$A_{22} \times B_{22}$	

Algoritmo de Fox. Ejemplo

P_{00}			P_{01}			P_{02}		
A_{00}	$A_{01} \times B_{10}$		A_{01}	$A_{01} \times B_{11}$		A_{02}	$A_{01} \times B_{12}$	
P_{10}			P_{11}			P_{12}		
A_{10}	$A_{12} \times B_{20}$		A_{11}	$A_{12} \times B_{21}$		A_{12}	$A_{12} \times B_{22}$	
P_{20}			P_{21}			P_{22}		
A_{20}	$A_{20} \times B_{00}$		A_{21}	$A_{20} \times B_{01}$		A_{22}	$A_{20} \times B_{02}$	

Algoritmo de Fox. Ejemplo

P_{00}			P_{01}			P_{02}		
A_{00}	$A_{02} \times B_{20}$		A_{01}	$A_{02} \times B_{21}$		A_{02}	$A_{02} \times B_{22}$	
P_{10}			P_{11}			P_{12}		
A_{10}	$A_{10} \times B_{00}$		A_{11}	$A_{10} \times B_{01}$		A_{12}	$A_{10} \times B_{02}$	
P_{20}			P_{21}			P_{22}		
A_{20}	$A_{21} \times B_{10}$		A_{21}	$A_{21} \times B_{11}$		A_{22}	$A_{21} \times B_{12}$	

Algoritmo de Strassen. Ejemplo

$$\begin{pmatrix} a_{11} & a_{12} \\ a_{21} & a_{22} \end{pmatrix} \begin{pmatrix} b_{11} & b_{12} \\ b_{21} & b_{22} \end{pmatrix}$$

$$p_1 = (a_{11} + a_{22})(b_{11} + b_{22})$$

$$p_2 = (a_{21} + a_{22})(b_{11})$$

$$p_3 = (a_{11})(b_{12} - b_{22})$$

$$p_4 = (a_{22})(-b_{11} + b_{21})$$

$$p_5 = (a_{11} + a_{12})(b_{22})$$

$$p_6 = (-a_{11} + a_{21})(b_{11} + b_{12})$$

$$p_7 = (a_{12} - a_{22})(b_{21} + b_{22})$$

$$\begin{pmatrix} p_1 + p_4 - p_5 + p_7 & p_3 + p_5 \\ p_2 + p_4 & p_1 + p_2 - p_3 + p_6 \end{pmatrix}$$

8 multiplicaciones

4 sumas

7 multiplicaciones

18 sumas

Algoritmo de Strassen. Ejemplo

Strassen itera de forma recursiva hasta su caso base (matriz 2x2).

$$\left(\begin{array}{cc|cc} 1 & 3 & 2 & -1 \\ 2 & 0 & 1 & 0 \\ \hline 9 & 2 & 8 & 4 \\ 4 & 1 & 0 & 2 \end{array} \right)$$

Temporización

```
105     MPI_Barrier(MPI_COMM_WORLD);
106     double t0 = MPI_Wtime();
107
108     for (int i = 0; i < rows_per_proc; ++i) {
109         for (int k = 0; k < n; ++k) {
110             int a_ik = local_A[i * n + k];
111             for (int j = 0; j < n; ++j) {
112                 local_C[i * n + j] += a_ik * B[k * n + j];
113             }
114         }
115     }
116
117     double local_elapsed = MPI_Wtime() - t0;
118
```

RESULTADOS

Tablas, tiempo de
ejecución, speed-up

04

Tabla

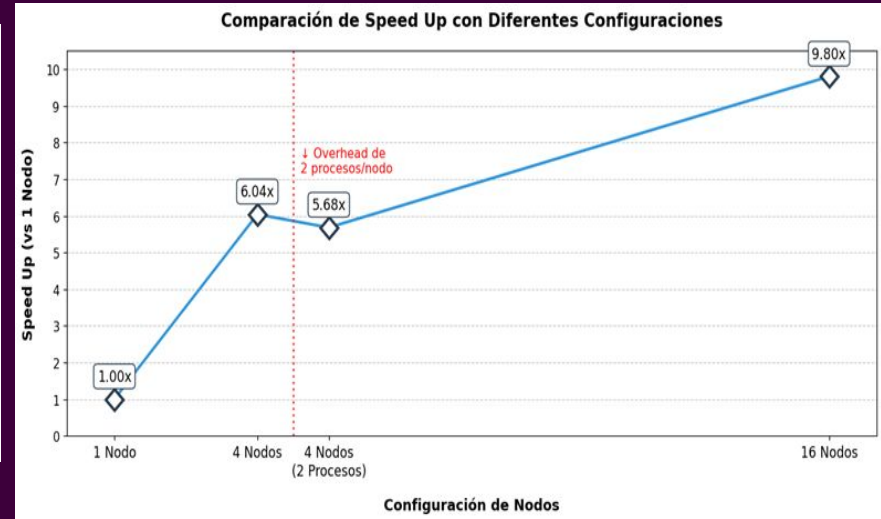
TIEMPOS MEDIOS DE ALGORITMOS DE MULTIPLICACIÓN

Algoritmos	<u>1 Nodos</u>	4 Nodos	<u>4 Nodos</u> (2 procesos)	<u>8 Nodos</u>	<u>16 Nodos</u>
Fox	11.604138	1.926094	<u>2.042579</u>	-	1.18907
Cannon	10.740777	2.114	2.092756	-	1.076616
Block row-wise	7.424116	1.568753	1.682913	1.004366	0.692126

Tiempo de ejecución. Speed-up

ALGORITMO DE FOX

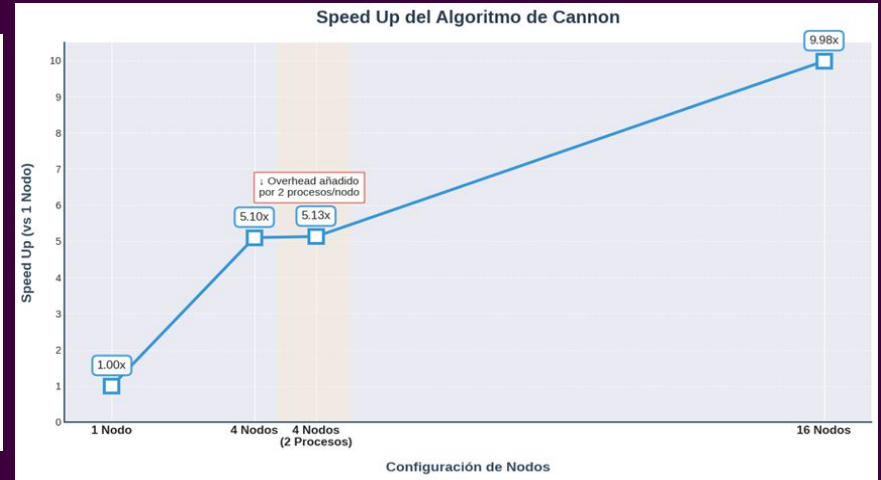
Métrica	1 Nodo	4 <u>Nodos</u>	4 <u>Nodos</u> (2procesos)	16 <u>Nodos</u>
Tiempo medio	11.604138	1.926094	2.042579	1.18907
Tiempo más alto	12.887646	2.154105	2.107730	1.251741
Tiempo más bajo	10.477805	1.790961	1.910982	1.148158
Variación	2.409841	0.363144	0.196748	0.103583
Speed up	1.0	6.04	5.68	9.8



Tiempo de ejecución. Speed-up

ALGORITMO DE CANNON

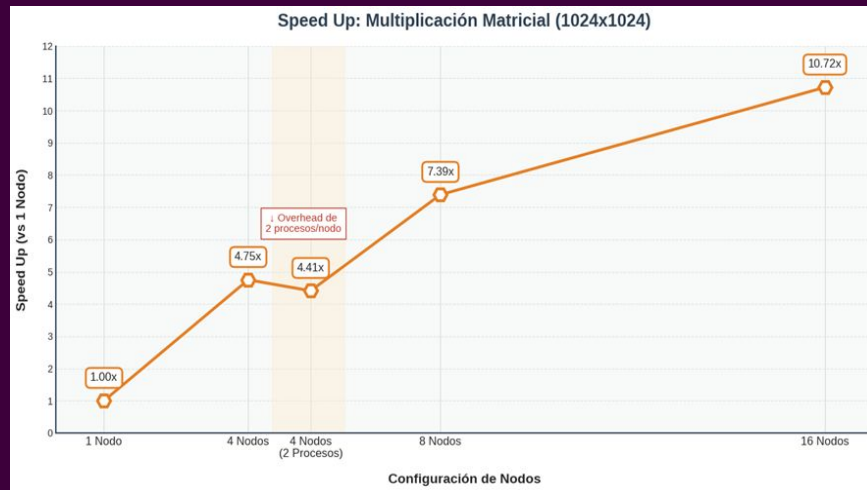
Métrica	1 <u>Nodo</u>	4 <u>Nodos</u>	4 <u>Nodos</u> (2procesos)	16 <u>Nodos</u>
Tiempo medio	10.740777	2.114	2.092756	1.076616
Tiempo más alto	11.789626	2.234247	2.192979	1.180969
Tiempo más bajo	8.009808	1.881399	1.992775	0.945468
Variación	3.779818	0.352848	0.200204	0.235501
Speed up	1.0	5.1	5.13	9.98



Tiempo de ejecución. Speed-up

ALGORITMO DE BLOCK ROW-WISE

Métrica	1 <u>Nodo</u>	4 <u>Nodos</u>	4 <u>Nodos</u> (2procesos)	8 <u>Nodos</u>	16 <u>Nodos</u>
Tiempo medio	7.424116	1.568753	1.682913	1.004366	0.692126
Tiempo más alto	8.581758	1.675503	2.000700	1.083501	0.705126
Tiempo más bajo	6.701251	1.453765	1.480423	0.890586	0.682067
Variación	1.880507	0.221738	0.520277	0.192915	0.023059
Speed up	1.0	4.75	4.41	7.39	10.72



DISCUSIÓN

05

Ventajas y Desventajas

Algoritmo	Ventajas	Desventajas
Strassen	• Reduce la complejidad teórica de la multiplicación ($\approx O(n^2.81)$ vs $O(n^3)$) • Puede ser más rápido para matrices grandes	• Mayor complejidad en la implementación distribuida • Comunicación intensa • Más memoria requerida • No tan eficiente para matrices pequeñas o bloques muy pequeños
Cannon	• Buena escalabilidad con procesos organizados en malla 2D • Comunicación estructurada y predecible • Balance de carga homogéneo	• Requiere número de procesos como cuadrado perfecto • Coste inicial de alineación • No aprovecha bien arquitecturas no regulares
Fox	• Similar a Cannon pero con mejor paralelismo en la comunicación • Más flexible que Cannon para ciertas arquitecturas	• También requiere cuadrado perfecto de procesos • Más complejidad en la gestión de comunicación • Coste adicional en desplazamiento inicial
Block Row-wise	• Muy simple de implementar • Escala bien con número creciente de procesos • Menos sobrecarga de sincronización	• Comunicación no optimizada (broadcast para bloques completos) • Menos eficiente en escalabilidad fina • Puede generar cuellos de botella en comunicación

Conclusión

06

Conclusión

- **Strassen:** Destaca por su menor complejidad matemática, pero su implementación y comunicación son más complejas y costosas en memoria.
- **Cannon:** Es un algoritmo con buena escalabilidad en sistemas con número de procesos que sea un cuadrado perfecto; ofrece un buen equilibrio entre complejidad y comunicación.
- **Fox:** Similar a Cannon en escalabilidad, pero con mayor paralelismo en comunicación, lo que puede mejorar el rendimiento a costa de una mayor complejidad de implementación.
- **Block Row-wise:** Es el más sencillo de implementar y mantiene una buena escalabilidad básica, aunque su eficiencia y uso de comunicación son menos óptimos en sistemas muy grandes o con comunicación fina.

Introducción a la programación distribuida

con MPICH

Navajas Román, Ángel - i02naroa
Sanchez Gutierrez, Daniel